Introduction to FetchType in JPA

Introduction

When working with object-relational mapping (ORM) frameworks like JPA (Java Persistence API) or Hibernate, it's essential to understand how related entities are loaded from the database. JPA provides two fetching strategies: eager and lazy loading, defined using the FetchType property. These fetching strategies control when related entities are retrieved along with the main entity.

What is FetchType?

FetchType in JPA/Hibernate defines when related entities are loaded from the database. There are two main fetching types:

- **Eager Fetching (FetchType.EAGER)**: Related entities are fetched immediately along with the main entity.
- **Lazy Fetching (FetchType.LAZY)**: Related entities are fetched on demand (when accessed for the first time).

Where is it used?

The fetch type is used in annotations that define relationships between entities such as @OneToOne, @ManyToOne, @OneToMany, and @ManyToMany.

```
@ManyToOne(fetch = FetchType.LAZY)  // Lazy fetch example
private Doctor doctor;
```

FetchType.EAGER

With FetchType.EAGER, related entities are loaded immediately when the parent entity is fetched from the database. This is the default fetch type for @ManyToOne and @OneToOne relationships.

Example Scenario: Patients and Doctors

Let's say we have two entities: Patient and Doctor. Each patient is associated with one doctor.

```
@Entity
public class Patient {
```

```
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    @ManyToOne(fetch = FetchType.EAGER)  // Eager fetching
    private Doctor doctor;
}
```

Here, when we fetch a Patient entity, the related Doctor entity is fetched immediately, regardless of whether we use the doctor field or not.

Advantages of Eager Fetching:

- **Simple Access**: You don't have to worry about loading related entities later.
- **No LazyInitializationException**: Because the related entities are loaded with the parent, you won't run into issues when accessing them later.

FetchType.LAZY

With FetchType.LAZY, related entities are not fetched until they are accessed for the first time. This is the default fetch type for @OneToMany and @ManyToMany relationships.

Example Scenario: Lazy Loading Doctor for Patient

```
@Entity
public class Patient {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    @ManyToOne(fetch = FetchType.LAZY)  // Lazy fetching
    private Doctor doctor;
}
```

With lazy fetching, when a Patient is retrieved, the Doctor is not immediately loaded. The Doctor will only be fetched when the doctor field is accessed.

Advantages of Lazy Fetching:

- **Performance Optimization**: Only the data you need is fetched from the database, avoiding unnecessary queries.
- **Memory Efficiency**: The database fetches less data initially, reducing memory consumption.

Common Issue: LazyInitializationException

What is it?

LazyInitializationException happens when you try to access a lazy-loaded entity outside of a transaction or session. This typically occurs in the service layer or controller after the database session is closed.

Scenario:

Let's say you have the following service method:

```
@Service
public class PatientService {

@Autowired
private PatientRepository patientRepository;

public String getDoctorName(Long patientId) {

// Doctor is not fetched yet
Patient patient = patientRepository.findById(patientId);

// LazyInitializationException might occur here
return patient.getDoctor().getName();
    }
}
```

If this method is called in a controller, and the session is already closed when accessing the doctor field, a LazyInitializationException will occur.

Conclusion

Choosing the right fetch type (EAGER or LAZY) is crucial for balancing performance and simplicity in your Spring Data JPA applications.

- **FetchType.EAGER**: Related entities are fetched immediately. It's useful when you always need the related data, but it can lead to performance problems if overused.

- **FetchType.LAZY**: Related entities are fetched only when accessed. This is ideal for performance optimisation, but be mindful of the potential for LazyInitializationException if accessed outside of an active session.

Understanding where to apply eager fetching can help you avoid common pitfalls like LazyInitializationException.

References

1. Eager & Lazy Loading
2. Overview of JPA/Hibernate Cascade Types